

Here's how to connect your fuzzy search from the client to your backend. I'll provide a complete implementation with both frontend and backend integration:

1. Frontend: Search Hook with Debouncing

```
// app/hooks/useSearch.ts
'use client';

import { useState, useEffect, useCallback } from 'react';
import { debounce } from 'lodash'; // or create your own debounce

interface UseSearchOptions<T> {
  searchFn: (query: string) => Promise<T[]>;
  debounceMs?: number;
  minQueryLength?: number;
}

export function useSearch<T>({
  searchFn,
  debounceMs = 300,
  minQueryLength = 2
}: UseSearchOptions<T>) {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState<T[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const debouncedSearch = useCallback(
    debounce(async (searchQuery: string) => {
      if (searchQuery.length < minQueryLength) {
        setResults([]);
        setIsLoading(false);
        return;
      }

      setIsLoading(true);
      setError(null);

      try {
        const data = await searchFn(searchQuery);
        setResults(data);
      } catch (err) {
        setError(err instanceof Error ? err.message : 'Search failed');
        setResults([]);
      } finally {
        setIsLoading(false);
      }
    }, debounceMs),
    [searchFn, minQueryLength]
  );
}
```

```

useEffect(() => {
  debouncedSearch(query);

  return () => {
    debouncedSearch.cancel();
  };
}, [query, debouncedSearch]);

return {
  query,
  setQuery,
  results,
  isLoading,
  error,
  clearResults: () => setResults([]),
};
}

```

2. API Service Layer

```

// app/services/search.service.ts
import { Career } from '@app/dashboard/careers/_component/CareerTable';
import { Property } from '@app/dashboard/properties/_component/PropertyTable';

const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL ||
'http://localhost:3001/api';

interface SearchResponse<T> {
  data: T[];
  total: number;
  page: number;
  pageSize: number;
}

class SearchService {
  private async request<T>(
    endpoint: string,
    params: Record<string, string | number>
  ): Promise<T> {
    const queryString = new URLSearchParams(
      Object.entries(params).reduce((acc, [key, value]) => {
        acc[key] = String(value);
        return acc;
      }, {}) as Record<string, string>
    ).toString();

    const response = await fetch(`${API_BASE_URL}${endpoint}?${queryString}`, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        // Add auth token if needed

```

```

        // 'Authorization': `Bearer ${localStorage.getItem('token')}`,
    },
  });

  if (!response.ok) {
    throw new Error(`Search failed: ${response.statusText}`);
  }

  return response.json();
}

async searchCareers(query: string, page = 1, pageSize = 10):
Promise<SearchResponse<Career>> {
  return this.request<SearchResponse<Career>>('/search/careers', {
    q: query,
    page,
    pageSize,
    fuzzy: 'true', // Enable fuzzy search
  });
}

async searchProperties(query: string, page = 1, pageSize = 10):
Promise<SearchResponse<Property>> {
  return this.request<SearchResponse<Property>>('/search/properties', {
    q: query,
    page,
    pageSize,
    fuzzy: 'true',
  });
}

async globalSearch(query: string): Promise<{
  careers: Career[];
  properties: Property[];
}> {
  return this.request('/search', { q: query, fuzzy: 'true' });
}

export const searchService = new SearchService();

```

3. Search Component with Fuzzy Integration

```

// app/component/ui/SearchInput.tsx
'use client';

import React, { useState, useEffect } from 'react';
import { Search, X, Loader2 } from 'lucide-react';
import { Input } from '@app/component/ui';
import { cn } from '@app/lib/utils';
import { useSearch } from '@app/hooks/useSearch';

```

```
import { searchService } from '@app/services/search.service';
import Link from 'next/link';

interface SearchInputProps {
  type: 'careers' | 'properties' | 'global';
  placeholder?: string;
  className?: string;
  onResultSelect?: (result: any) => void;
}

export function SearchInput({
  type,
  placeholder = 'Search...',
  className,
  onResultSelect
}: SearchInputProps) {
  const [isFocused, setIsFocused] = useState(false);
  const [showResults, setShowResults] = useState(false);

  const getSearchFunction = () => {
    switch (type) {
      case 'careers':
        return (query: string) => searchService.searchCareers(query).then(r =>
r.data);
      case 'properties':
        return (query: string) => searchService.searchProperties(query).then(r =>
r.data);
      case 'global':
        return async (query: string) => {
          const results = await searchService.globalSearch(query);
          return [...results.careers, ...results.properties];
        };
      default:
        throw new Error('Invalid search type');
    }
  };

  const { query, setQuery, results, isLoading, error, clearResults } = useSearch({
    searchFn: getSearchFunction(),
    debounceMs: 300,
    minQueryLength: 2,
  });

  const handleFocus = () => {
    setIsFocused(true);
    if (query.length >= 2) {
      setShowResults(true);
    }
  };

  const handleBlur = () => {
    setTimeout(() => {
      setIsFocused(false);
      setShowResults(false);
    }, 100);
  };
}
```

5 / 11

```

        </div>
      ) : results.length > 0 ? (
        <div className="py-2">
          {results.map((result: any, index) => (
            <SearchResultItem
              key={result.id || index}
              result={result}
              type={type}
              onClick={() => handleResultClick(result)}
            />
          ))}
        </div>
      ) : (
        <div className="p-4 text-center text-gray-400">
          No results found for "{query}"
        </div>
      )}
    </div>
  )}
</div>
);
}

function SearchResultItem({ result, type, onClick }: any) {
  const getHref = () => {
    if (result.jobTitle) {
      return `/dashboard/careers/${result.id}`;
    }
    if (result.title) {
      return `/dashboard/properties/${result.id}`;
    }
    return '#';
  };

  return (
    <Link
      href={getHref()}
      onClick={onClick}
      className="block px-4 py-2 hover:bg-white/10 transition-colors"
    >
      <div className="text-white">
        {result.jobTitle || result.title}
      </div>
      <div className="text-sm text-gray-400">
        {result.location || result.employmentType || result.status}
      </div>
    </Link>
  );
}

```

4. Backend API Implementation (Node.js/Express with Fuse.js)

```
// backend/routes/search.js
const express = require('express');
const router = express.Router();
const Fuse = require('fuse.js');

// Sample data - replace with your database queries
const careers = [
  { id: '1', jobTitle: 'Software Engineer', location: 'Remote', employmentType: 'Full Time' },
  { id: '2', jobTitle: 'Product Manager', location: 'Lagos', employmentType: 'Contract' },
  // ... more data
];

const properties = [
  { id: '1', title: 'Luxury Apartment', location: 'Lekki', price: 50000000 },
  { id: '2', title: 'Office Space', location: 'Victoria Island', price: 120000000 },
  // ... more data
];

// Fuzzy search options
const fuseOptions = {
  includeScore: true,
  threshold: 0.4, // Lower = stricter matching
  minMatchCharLength: 2,
  keys: [
    { name: 'jobTitle', weight: 0.7 },
    { name: 'location', weight: 0.5 },
    { name: 'employmentType', weight: 0.3 },
  ]
};

// Search careers endpoint
router.get('/careers', (req, res) => {
  const { q, page = 1, pageSize = 10, fuzzy = 'true' } = req.query;

  if (!q) {
    return res.json({ data: [], total: 0, page: 1, pageSize });
  }

  let results;

  if (fuzzy === 'true') {
    const fuse = new Fuse(careers, {
      ...fuseOptions,
      keys: ['jobTitle', 'location', 'employmentType'],
    });
    results = fuse.search(q).map(r => r.item);
  } else {
    // Exact search fallback
    const lowerQ = q.toLowerCase();
```

```
    results = careers.filter(c =>
      c.jobTitle.toLowerCase().includes(lowerQ) ||
      c.location.toLowerCase().includes(lowerQ)
    );
  }

  // Pagination
  const start = (page - 1) * pageSize;
  const end = start + parseInt(pageSize);
  const paginatedResults = results.slice(start, end);

  res.json({
    data: paginatedResults,
    total: results.length,
    page: parseInt(page),
    pageSize: parseInt(pageSize),
  });
});

// Search properties endpoint
router.get('/properties', (req, res) => {
  const { q, page = 1, pageSize = 10, fuzzy = 'true' } = req.query;

  if (!q) {
    return res.json({ data: [], total: 0, page: 1, pageSize });
  }

  const fuse = new Fuse(properties, {
    includeScore: true,
    threshold: 0.4,
    keys: ['title', 'location'],
  });

  const results = fuzzy === 'true'
    ? fuse.search(q).map(r => r.item)
    : properties.filter(p =>
      p.title.toLowerCase().includes(q.toLowerCase()) ||
      p.location.toLowerCase().includes(q.toLowerCase())
    );

  const start = (page - 1) * pageSize;
  const end = start + parseInt(pageSize);
  const paginatedResults = results.slice(start, end);

  res.json({
    data: paginatedResults,
    total: results.length,
    page: parseInt(page),
    pageSize: parseInt(pageSize),
  });
});

// Global search endpoint
router.get('/', (req, res) => {
```



```

const { q, fuzzy = 'true' } = req.query;

if (!q) {
  return res.json({ careers: [], properties: [] });
}

const careerFuse = new Fuse(careers, {
  threshold: 0.4,
  keys: ['jobTitle', 'location'],
});

const propertyFuse = new Fuse(properties, {
  threshold: 0.4,
  keys: ['title', 'location'],
});

const careerResults = fuzzy === 'true'
  ? careerFuse.search(q).map(r => r.item).slice(0, 5)
  : careers.filter(c =>
    c.jobTitle.toLowerCase().includes(q.toLowerCase())
  ).slice(0, 5);

const propertyResults = fuzzy === 'true'
  ? propertyFuse.search(q).map(r => r.item).slice(0, 5)
  : properties.filter(p =>
    p.title.toLowerCase().includes(q.toLowerCase())
  ).slice(0, 5);

res.json({
  careers: careerResults,
  properties: propertyResults,
});
});

module.exports = router;

```

5. Usage in Your Dashboard

```

// app/dashboard/layout.tsx or app/dashboard/page.tsx
'use client';

import { SearchInput } from '@app/component/ui/SearchInput';

export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return (
    <div>
      <header className="flex items-center justify-between p-4 border-b border-white/10">
        <div className="flex-1 max-w-md">
          <SearchInput

```

```

        type="global"
        placeholder="Search careers, properties..."
      />
    </div>
  </header>
  {children}
</div>
);
}

```

6. Advanced: PostgreSQL Full-Text Search with Fuzzy Matching

If using PostgreSQL, you can implement fuzzy search with trigrams:

```

-- Enable trigram extension
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Create index for faster fuzzy search
CREATE INDEX careers_search_idx ON careers USING GIN (
  (job_title || ' ' || location || ' ' || employment_type) gin_trgm_ops
);

-- Search query with similarity ranking
SELECT
  *,
  similarity(job_title || ' ' || location, $1) as rank
FROM careers
WHERE
  job_title % $1
  OR location % $1
ORDER BY rank DESC
LIMIT $2;

```

```

// backend/services/search.service.js
const { Pool } = require('pg');

class SearchService {
  constructor() {
    this.pool = new Pool({
      // your database config
    });
  }

  async fuzzySearchCareers(query, limit = 10) {
    const sql = `
      SELECT
        *,
        GREATEST(
          similarity(job_title, $1),

```

```
        similarity(location, $1)
      ) as rank
    FROM careers
    WHERE
      job_title % $1
      OR location % $1
      OR employment_type % $1
    ORDER BY rank DESC
    LIMIT $2
  `;

  const result = await this.pool.query(sql, [query, limit]);
  return result.rows;
}
}
```

This implementation provides:

1. **Debounced search** to avoid excessive API calls
2. **Fuzzy matching** on both frontend (optional) and backend
3. **Pagination support** for large result sets
4. **Type safety** with TypeScript
5. **Reusable hooks and components**
6. **Multiple search strategies** (global vs specific)

The backend can be swapped between in-memory Fuse.js for development and PostgreSQL full-text search for production.